

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA061

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 10

Total Marks:50

Annexure A

DEBUGGING & OPTIMISATION TASK

Code of Conduct:

I Yaswanthika Shree S (192565069) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Yaswanthika Shree S

Annexure A

DEBUGGING & OPTIMISATION TASK

Q36. Debugging Convex Hull Logic (Collinear Case Mishandling)

The following brute force Convex Hull edge validation logic fails in cases where all remaining points are collinear with a candidate edge. Carefully analyze how collinear points should be handled and explain why the current condition may reject valid hull edges or include incorrect ones. Debug the logic to correctly distinguish between boundary edges and invalid interior segments. Optimize the implementation by reducing repeated orientation checks if possible. Analyze the time complexity of the corrected brute force solution and present the improved code clearly.

```
def is_hull_edge(p1, p2, points):
    for p in points:
        val = orientation(p1, p2, p)
        if val == 0:
            return False # ERROR
        if val < 0:
            return False
    return True
    while j >= 0 and arr[j] > key:
        arr[j+1] = key # ERROR
        arr[j] = arr[j]
        j -= 1
    return arr
```

```
Programiz C Online Compiler
main.c
1 #include <stdio.h>
2 int orientation(float p1[2], float p2[2], float p[2]) {
3     float val = (p2[0]-p1[0])*(p[1]-p1[1]) - (p2[1]-p1[1])*(p[0]-p1[0]);
4     if (val > 0) return 1;
5     else if (val < 0) return -1;
6     else return 0;
7 }
8 int is_hull_edge(float p1[2], float p2[2], float points[][2], int n) {
9     int left = 0, right = 0;
10    for (int i = 0; i < n; i++) {
11        float p[2] = {points[i][0], points[i][1]};
12        if ((p[0]==p1[0] && p[1]==p1[1]) || (p[0]==p2[0] && p[1]==p2[1]))
13            continue;
14        int result = orientation(p1, p2, p);
15        if (result == 1) left = 1;
16        else if (result == -1) right = 1;
17    }
18    return left && right;
19 }
20 int main() {
21     float points[][2] = {{0,0}, {4,0}, {0,3}, {4,3}, {4,0}, {4,3}, {4,3}, {0,3}, {0,3}, {4,3}, {2,3}, {0,3}, {2,3}, {0,3}, {2,3}, {0,3}, {2,3}, {0,3}, {2,3}};
22     int n = 20;
23     float p1[2] = {0,0};
24     float p2[2] = {4,0};
25     if (is_hull_edge(p1, p2, points, n))
26         printf("All points are on the hull edge.\n");
27     else
28         printf("Not all points are on the hull edge.\n");
29     return 0;
30 }
```

Output

```
Checking all possible edges:
(0,0) - (4,0) => HULL EDGE
(0,0) - (0,3) => HULL EDGE
(4,0) - (0,0) => HULL EDGE
(4,0) - (4,3) => HULL EDGE
(4,3) - (4,0) => HULL EDGE
(4,3) - (0,3) => HULL EDGE
(4,3) - (2,3) => HULL EDGE
(0,3) - (0,0) => HULL EDGE
(0,3) - (4,3) => HULL EDGE
(0,3) - (2,3) => HULL EDGE
(2,3) - (4,3) => HULL EDGE
(2,3) - (0,3) => HULL EDGE
=== Code Execution Successful ===
```

```

Programiz C Online Compiler
main.c
16     else if (result == -1) right = 1;
17     }
18     return !(left && right);
19 }
20 int main() {
21     float points[6][2] = {{0,0}, {4,0}, {4,3}, {0,3}, {2,3}, {2,1}};
22     ;
23     int n = 6;
24     printf("Checking all possible edges:\n\n");
25     for (int i = 0; i < n; i++) {
26         for (int j = 0; j < n; j++) {
27             if (i != j) {
28                 if (is_hull_edge(points[i], points[j], points, n)) {
29                     printf("(%.0f,%.0f) - (%.0f,%.0f) => HULL\n",
30                         points[i][0], points[i][1], points[j][0],
31                         points[j][1]);
32                 }
33             }
34         }
35     }
36 }

```

Output

```

Checking all possible edges:
(0,0) - (4,0) => HULL EDGE
(0,0) - (0,3) => HULL EDGE
(4,0) - (0,0) => HULL EDGE
(4,0) - (4,3) => HULL EDGE
(4,3) - (4,0) => HULL EDGE
(4,3) - (0,3) => HULL EDGE
(4,3) - (2,3) => HULL EDGE
(0,3) - (0,0) => HULL EDGE
(0,3) - (4,3) => HULL EDGE
(0,3) - (2,3) => HULL EDGE
(2,3) - (4,3) => HULL EDGE
(2,3) - (0,3) => HULL EDGE

```

=== Code Execution Successful ===

```

Programiz C Online Compiler
main.c
21     float points[6][2] = {{0,0}, {4,0}, {4,3}, {0,3}, {2,3}, {2,1}};
22     ;
23     int n = 6;
24     printf("Checking all possible edges:\n\n");
25     for (int i = 0; i < n; i++) {
26         for (int j = 0; j < n; j++) {
27             if (i != j) {
28                 if (is_hull_edge(points[i], points[j], points, n)) {
29                     printf("(%.0f,%.0f) - (%.0f,%.0f) => HULL\n",
30                         points[i][0], points[i][1], points[j][0],
31                         points[j][1]);
32                 }
33             }
34         }
35     }
36     return 0;
37 }

```

Output

```

Checking all possible edges:
(0,0) - (4,0) => HULL EDGE
(0,0) - (0,3) => HULL EDGE
(4,0) - (0,0) => HULL EDGE
(4,0) - (4,3) => HULL EDGE
(4,3) - (4,0) => HULL EDGE
(4,3) - (0,3) => HULL EDGE
(4,3) - (2,3) => HULL EDGE
(0,3) - (0,0) => HULL EDGE
(0,3) - (4,3) => HULL EDGE
(0,3) - (2,3) => HULL EDGE
(2,3) - (4,3) => HULL EDGE
(2,3) - (0,3) => HULL EDGE

```

=== Code Execution Successful ===

1.orientation() function

This checks where a point p lies compared to the line joining p1 and p2:

Returns 1 → point is on the left side

Returns -1 → point is on the right side

Returns 0 → point lies exactly on the line (collinear)

It does this using a simple cross-product formula.

2. is_hull_edge() function

This checks if the line p1-p2 can be an edge of the convex hull.

It loops through every point in the list.

Skips p1 and p2 themselves (no need to compare a point with itself).

For every other point, it calls orientation():

If the point is on the left → set left = 1

If the point is on the right → set right = 1

If the point is collinear (result == 0) → do nothing, just skip it (this is the bug fix — collinear points should not cancel the edge)

At the end:

If points were found on both sides (left = 1 and right = 1), the line passes through the middle of the shape → not a hull edge.

If points were found on only one side (or none, or just collinear points) → it is a valid hull edge.

3. main() function

Stores 6 points in a 2D array.

Uses two loops to try every possible pair of points as a candidate edge.

For each pair, it calls `is_hull_edge()`.

If the function says "yes, this is a hull edge," it prints that pair.

4. Why this fixes the original bug

The original code said: "if even one point is collinear, reject the edge." That was wrong, because collinear points are perfectly normal on a hull boundary (e.g., 3 points lying on the same straight edge of a rectangle). The fixed code instead says: "only reject the edge if points exist on both sides — collinear points don't count against it."

5. Time complexity

Outer two loops → checks every pair of points → $O(n^2)$

Inner loop inside `is_hull_edge()` → checks all n points again → $O(n)$

Total → $O(n^3)$, which is expected for a brute-force convex hull algorithm

RUBRICS FOR CALCULATION

DEBUGGING & OPTIMISATION TASK					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

